

Competitive Programming HandsOn N.3

Matteo Giuntori

September 2026

1 Holiday Planning

To solve this problem while adhering to the required computational complexity of $\mathcal{O}(nD^2)$ (where n represents the number of cities and D the total number of available holiday days), it is necessary to use a **Dynamic Programming** approach. This paradigm allows us to optimize the solution by building it upon the results of already computed subproblems.

First, for each city, we calculate the cumulative total number of attractions that can be visited by spending a specific number of days. Since the order of the itinerary is fixed and "cherry-picking" the absolute best single days is not allowed, the total number of visited attractions by spending k days in a specific city will correspond to the prefix sum up to day k . To minimize memory usage, we compute these sums on-the-fly for the current city using a single array `psum` of size $D + 1$, rather than storing them for all cities simultaneously.

Next, rather than allocating a full two-dimensional table, we maintain a 1D DP array of size $D + 1$. Each element $DP[j]$ represents the maximum number of attractions we can visit with j available days ($0 \leq j \leq D$). For each city, we update the array in reverse order (j from D down to 1) to preserve the values of the previous step:

$$DP[j] \leftarrow \max_{0 \leq k \leq j} (DP[j - k] + psum[k])$$

```
let mut dp = vec![0; d + 1];
let mut psum = vec![0; d + 1];

for _ in 0..n {
    let mut sum = 0;
    for i in 1..=d {
        let num: i32 = values_in.next().unwrap()
                                .parse().unwrap();

        sum += num;
        psum[i] = sum;
    }
    for j in (1..=d).rev() {
        let mut best = dp[j];
        for k in 1..=j {
            best = cmp::max(best, dp[j - k] + psum[k]);
        }
        dp[j] = best;
    }
}
```

This means that, to update the optimal value for j total days, we evaluate all possible daily distributions for the current city. By iterating over a number of days k (ranging from 0 to j) to dedicate to the current city, we calculate the sum of:

- $DP[j - k]$: the maximum number of attractions visited in the previously processed cities using the remaining $j - k$ days.
- $psum[k]$: the number of attractions visited by spending exactly k days in the current city.

Traversing j in descending order ensures that $DP[j - k]$ still holds the optimal value from the previous city, avoiding unwanted overwrites. At the end of the execution, the final solution to the problem will be located in $DP[D]$.

Complexity Analysis

In terms of **Time Complexity**, computing the prefix sums requires $\mathcal{O}(D)$ operations per city, totaling $\mathcal{O}(nD)$. During the Dynamic Programming phase, for each of the n cities and for each day j from 1 to D , the inner loop executes j steps. Since $\sum_{j=1}^D j \approx \frac{D^2}{2}$, the total computational cost remains dominated by the nested loops, resulting in exactly $\mathcal{O}(nD^2)$.

Regarding the **Space Complexity**, the requirement is optimized to $\mathcal{O}(D)$. Instead of allocating full $\mathcal{O}(nD)$ matrices, we only maintain the 1D array `dp` and the reusable buffer `psum`, each of size $D + 1$. Since there are no recursive calls, the stack space is negligible, keeping the spatial complexity strictly bound to $\mathcal{O}(D)$.

2 Design Course

In this section, I will explain the structure of the implemented solution to solve the problem with a time complexity of $\mathcal{O}(n \log n)$, where n represents the number of available topics (or lectures). The approach is based on a combination of strategic sorting and **Dynamic Programming**.

The goal is to reduce the request to the classic **Longest Increasing Subsequence (LIS)** problem. The main difficulty lies in having to ensure a strictly increasing monotonicity over two properties (beauty b and difficulty d), rather than just one. To overcome this obstacle, the solution involves stabilizing one of the two parameters by performing an initial *sorting*.

A fundamental detail concerns the sorting criterion: the list must be ordered in strictly increasing order with respect to the first value (beauty b), but, for equal beauty values, in **decreasing** order based on difficulty. This is because the problem requires a strictly increasing sequence for both b and d . If we were to sort d in increasing order for identical values of b , the LIS algorithm would end up selecting multiple topics with the same beauty, violating the constraint. By sorting decreasingly on d , we guarantee that the subsequent algorithm is forced to select at most one topic for each beauty level.

```
courses.sort_unstable_by(|a, b| {  
    a.0.cmp(&b.0).then(b.1.cmp(&a.1))  
});
```

Once this pre-processing is done, it is sufficient to apply the LIS algorithm exclusively to the other value. Having kept the beauty b fixed via sorting, we proceed with applying the pattern to the difficulties d .

To compute the LIS while maintaining logarithmic complexity, we initialize a support array called **tails**. The element at position i of **tails** stores the smallest possible value that acts as the final element (the "tail") for an increasing subsequence of length $i + 1$.

Iterating over the difficulties, we use binary search via the **partition_point** function to find the index of the first element in **tails** that is greater than or equal to the current difficulty. If this index coincides with the array's length, it means the current element is strictly greater than all tails recorded so far: we can therefore append it to **tails**, increasing the length of our LIS. Otherwise, we overwrite the value found at **idx** with the current difficulty. This replacement operation lowers the terminal value of a subsequence of that specific length, thereby maximizing the probability of being able to extend the sequence with future values.

```

let mut tails: Vec<i32> = Vec::new();

for (_, d) in courses {
    let idx = tails.partition_point(|&val| val < d);
    if idx == tails.len() {
        tails.push(d);
    } else {
        tails[idx] = d;
    }
}

```

Complexity Analysis

In terms of **Time Complexity**, the algorithm is divided into two main computational phases. The first phase consists of the initial sorting of the courses (performed via `sort_unstable_by`), an operation requiring a time of $\mathcal{O}(n \log n)$, where n represents the total number of available topics in the input. During the subsequent Dynamic Programming phase to search for the *Longest Increasing Subsequence*, we iterate linearly over all n elements of the sorted array. For each element, a binary search (via the `partition_point` method) is performed on the support array `tails`. Since the binary search has a logarithmic cost of $\mathcal{O}(\log n)$ and is repeated n times, the computational cost of this loop also amounts to $\mathcal{O}(n \log n)$. Consequently, the total time complexity is dominated by these two operations, resulting in exactly $\mathcal{O}(n \log n)$.

In terms of **Space Complexity**, the algorithm requires storing the n initial pairs in memory within a vector. To this, we add the memory allocated for the support array `tails`, whose maximum size in the worst case (i.e., when all courses form a strictly increasing sequence) will also be equal to n . Therefore, the overall memory footprint grows linearly with the input, settling at $\mathcal{O}(n)$.